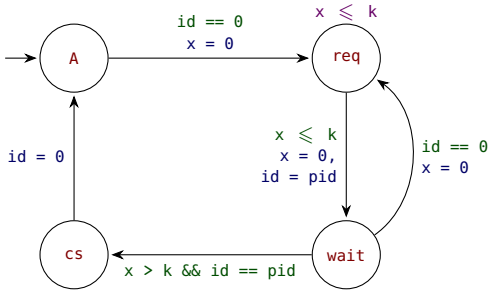
# Case Study: LLM-Assisted Exploration of Fischer's Protocol in Uppaal

Version 2, 9 October 2026

**Setting.** A user new to model checking asked an LLM assistant (Claude, connected to Uppaal through an MCP tool interface) to explain the model open in the editor: Fischer's mutual exclusion protocol with six processes `P(1)` … `P(6)`, a shared variable `id`, a local clock `x` and constant `k` = 2. Over one session the assistant explained the model, ran experiments on request, interpreted counterexamples, and restored the model after each change. The session used about 50 tool calls, 11 of them verification runs. User prompts were often minimal ("1", "2", "experiment 3") and referred to a numbered list of experiments the assistant had proposed.




| Variant | mutex | no deadlock | req⇝wait | req⇝cs |
|---|---|---|---|---|
| Original | ✓ | ✓ | ✓ | × (4, "lazy") |
| `x > k` → `x ⩾ k` | × (6) | – | – | – |
| no invariant on `req` | ✓ | × (6) | × (1) | – |
| eager variant | ✓ | ✓ | ✓ | × (15, lasso) |

Numbers give the length of the counterexample trace; – means not checked. The *eager variant* adds invariants on `wait` and `cs`, a `lost` location, and an `urgent broadcast chan retry;` channel that forces retries.

**Interesting points.**

1. **A one-character mutation exposes a boundary-instant race.** Weakening the guard on `wait`→`cs` from `x > k` to `x ⩾ k` breaks mutual exclusion. The shortest counterexample (6 steps, found in 0.007 s) needs one process to check at exactly $x = k$ while another is still allowed to write (`x ⩽ k`). Random simulation almost never hits this single instant; exhaustive symbolic exploration finds it at once.
2. **The checker corrected the assistant's prediction.** The assistant expected that removing the invariant `x ⩽ k` on `req` would break mutual exclusion. Verification showed that mutual exclusion still held, because the edge guard `x ⩽ k` remains, while `A[] not deadlock` and `P(1).req --> P(1).wait` failed. The explanation the assistant then gave, *guards permit, invariants force*, came from the tool's result, not from its own prior reasoning. LLM explanations should be grounded in mechanised checks.
3. **The first counterexample was not the one the user expected.** The fairness property `P(1).req --> P(1).cs` failed with a 4-step trace in which `P(1)` had *won* the race (`id == 1`) but could stay in `wait` forever, because `wait` has no invariant. To show genuine starvation, the assistant refined the model so that every process must act promptly. The result was a 15-step infinite lasso in which `P(5)` and `P(6)` win alternate rounds while `P(1)` always loses. A failing liveness property says *some* witness exists; finding the intended one may require changing the model.
4. **Safety and liveness separate clearly.** The ⩾ mutation broke safety, while removing the invariant broke liveness and deadlock freedom but left safety intact. This made the safety/liveness distinction concrete for the novice user: a system that never acts is trivially safe.
5. **Keeping the model consistent mattered in practice.** The assistant restored the model and re-verified after every experiment. Before restoring, it detected that the user had edited the model and checked that the edit was only saved results. It also noted that replacing the whole model discards the editor's undo history. Running experiments on a shared, live model needs this kind of care.
6. **The assistant could not see the GUI.** It had no view of the simulator, so for trace inspection it gave the user step-by-step instructions instead. Counterexamples were therefore explained in tables built from the symbolic traces, with clock zones such as `P(1).x` $\in [0, 2]$ rather than concrete timings.

**Takeaway.** Combining an LLM with a model checker worked best when the checker was the arbiter. The LLM proposed experiments, carried out edits and translated traces into plain language, while every claim about the model's behaviour was settled by verification, including the one where the LLM was wrong.